Kubernetes on AWS

Complete Beginner's Guide -- Architecture . Commands . EKS

K8s v1.29+ . AWS EKS . eksctl . kubectl . Spring Boot . May 2025

1. What is Kubernetes?

Kubernetes (K8s) is a **container orchestration system** that automatically manages, scales, and keeps your containerised applications running. You tell Kubernetes *what you want* and it figures out *how to make it happen.*

> *"Kubernetes is like a restaurant manager. You say: I need 3 waiters at all times. The manager hires when someone quits, fires when it's slow, and moves staff around. You never manage individual people -- just the rules."*

The Problem It Solves

Running containers manually on VMs breaks when:

* Your app crashes at 3 AM -- no one restarts it
* Traffic spikes -- you cannot scale fast enough
* A server dies -- containers on that server are gone
* You deploy a new version -- downtime while swapping containers

Kubernetes solves all of these automatically.

Docker vs Kubernetes

	Docker	Kubernetes
Scale	One container, one machine	Thousands of containers across hundreds of machines
Self-healing	Manual restart if crash	Automatic restart in seconds
Scaling	Manual	Automatic -- CPU/memory/custom metrics
Updates	Manual -- downtime during swap	Rolling update -- zero downtime
Service discovery	Manual configuration	Built-in DNS for every service

Kubernetes on AWS -- Amazon EKS

Amazon EKS (Elastic Kubernetes Service) runs and manages Kubernetes for you:

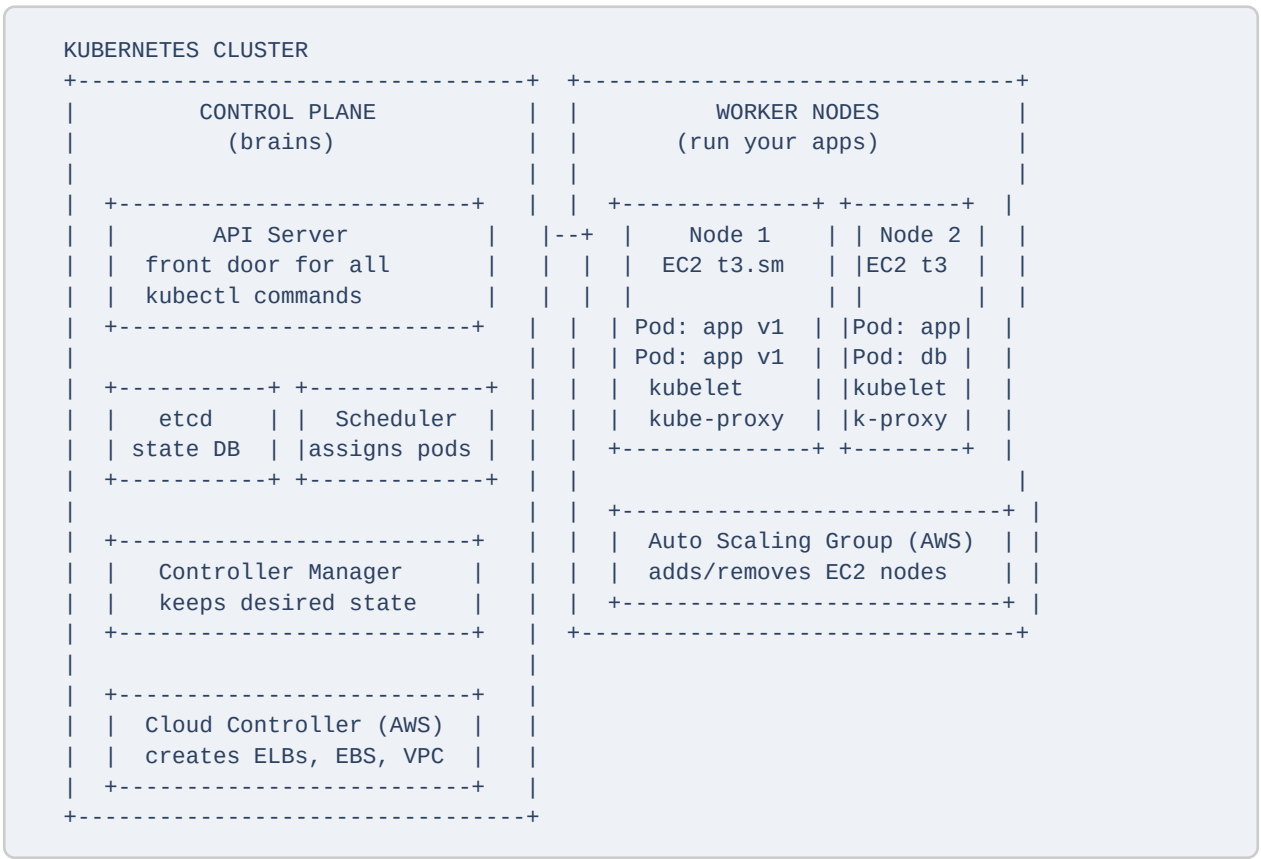
* AWS manages the **control plane** (the cluster brain) -- you never touch it
* You manage only the **worker nodes** (EC2 instances running your apps)
* EKS integrates with AWS: ELB, ECR, IAM, EBS, RDS, CloudWatch
* Cost: $0.10/hour for the control plane + EC2 instance costs for nodes

Why YAML Files?

You describe your **desired state** in YAML files. Kubernetes continuously reconciles actual state to match it. If 1 pod crashes -> K8s sees actual=2, desired=3 -> starts a new pod automatically.

---

2. Cluster Architecture -- The Two Planes



Control Plane Components

Component	Role
API Server	The front door. Every kubectl command and component communication goes through here
etcd	The cluster's database. Stores the entire desired state -- every resource definition

Component	Role
Scheduler	Decides which worker node each new pod runs on (resources, affinity, taints)
Controller Manager	Watches actual state and corrects drift -- restarts crashed pods, scales deployments
Cloud Controller (AWS)	Talks to AWS APIs -- creates ELBs for LoadBalancer Services, provisions EBS volumes

Worker Node Components

Component	Role
kubelet	Agent on every node. Talks to the API Server. Starts/stops pods as instructed
kube-proxy	Handles network rules on each node -- routes traffic to the right pod
Container runtime	Usually containerd. Actually runs Docker containers

---

3. Core Building Blocks

3.1 Pod -- The Smallest Unit

A Pod wraps one or more containers that always run together on the same node. They share the same network (IP address) and storage.

> *"A Pod is like a shipping container. Inside you put your app (and maybe a helper sidecar). The whole unit moves together."*

* Every pod gets its own unique IP address inside the cluster
* Pods are **ephemeral** -- they can die and be replaced at any time with a new IP
* You rarely create pods directly -- Deployments manage them for you

Minimal pod YAML:

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-app
  labels:
    app: my-app
spec:
  containers:
  - name: my-app
    image: nginx:latest
    ports:
    - containerPort: 80
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

Pod commands:

```bash
# Create a pod (for learning -- use Deployment in production)
kubectl run my-app --image=nginx:latest --port=80

# See all running pods
kubectl get pods

# Pods with node assignment and IP
kubectl get pods -o wide

# Detailed info including events (best debugging tool)
kubectl describe pod my-app

# Live logs (-f = follow)
kubectl logs -f my-app

# Shell into a running pod
kubectl exec -it my-app -- /bin/bash
```

3.2 Deployment -- Manages Pod Replicas

A Deployment says "always run N copies of this pod." It handles: self-healing (restarts crashed pods), rolling updates (zero downtime), and rollbacks (instant revert).

> *"A Deployment is a staffing contract: Always have 3 waiters. If one quits, hire another immediately."*

Complete deployment.yaml:

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  namespace: production
spec:
  replicas: 3                    # always run 3 pods
  selector:
    matchLabels:
      app: my-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1               # 1 extra pod during update
      maxUnavailable: 0         # never have 0 pods available
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: my-app
        image: 123456789.dkr.ecr.us-east-1.amazonaws.com/my-app:1.0.0
        ports:
        - containerPort: 8080
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"
        livenessProbe:
          httpGet:
            path: /actuator/health/liveness
            port: 8080
          initialDelaySeconds: 60
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /actuator/health/readiness
            port: 8080
          initialDelaySeconds: 30
          periodSeconds: 5
```

Deployment commands:

```
kubectl apply -f deployment.yaml
kubectl rollout status deployment/my-app

# Scale immediately
kubectl scale deployment my-app --replicas=6

# Zero-downtime update to new version
kubectl set image deployment/my-app my-app=my-app:2.0.0

# Watch pods rolling update in real time
kubectl get pods -w

# Rollback to previous version (instant)
kubectl rollout undo deployment/my-app

# See rollout history
kubectl rollout history deployment/my-app
```

3.3 Service -- Stable Network Endpoint

Pods get new IPs every time they are recreated. A Service gives you a stable DNS name and IP that never changes. It load-balances traffic across all matching pods automatically.

Type	Accessible from	When to use
ClusterIP (default)	Inside cluster only	Pod-to-pod communication
NodePort	Outside via node IP + port	Development/testing only
LoadBalancer	Internet via AWS ELB	Exposing your app to users

service.yaml

```
# service.yaml -- creates an AWS ELB automatically
apiVersion: v1
kind: Service
metadata:
  name: my-app-service
  namespace: production
spec:
  type: LoadBalancer          # AWS provisions an ELB automatically
  selector:
    app: my-app               # routes to pods with this label
  ports:
  - protocol: TCP
    port: 80                  # external port users call
    targetPort: 8080          # pod's internal port
```

```
# Get the public URL (takes 1-2 min to provision ELB)
kubectl get service my-app-service
# EXTERNAL-IP: abc123.us-east-1.elb.amazonaws.com

# Call from inside another pod using DNS name
curl http://my-app-service.production.svc.cluster.local/api/health
```

3.4 Namespace -- Virtual Cluster

Namespaces split one cluster into separate virtual environments -- dev, staging, prod -- with different teams and resource limits.

> *"A namespace is like a floor of an office building. Floor 1 = dev team. Floor 2 = prod team. They share the building but don't interfere with each other."*

```
# Create namespaces for each environment
kubectl create namespace dev
kubectl create namespace staging
kubectl create namespace production

# Deploy to a specific namespace
kubectl apply -f deployment.yaml -n production

# See all resources in a namespace
kubectl get all -n production

# Set default namespace for your session
kubectl config set-context --current --namespace=production
```

3.5 ConfigMap and Secret

Never hardcode config in your Docker image. ConfigMaps store non-sensitive config; Secrets store sensitive values (passwords, API keys).

ConfigMap -- non-sensitive configuration:

```
kubectl create configmap app-config \
  --from-literal=LOG_LEVEL=INFO \
  --from-literal=SERVER_PORT=8080

# Use in deployment.yaml:
env:
- name: LOG_LEVEL
  valueFrom:
    configMapKeyRef:
      name: app-config
      key: LOG_LEVEL
```

Secret -- sensitive values:

```
kubectl create secret generic db-secret \
  --from-literal=DB_PASSWORD=mypassword \
  --from-literal=JWT_SECRET=myjwtsecretkey

# Use in deployment.yaml:
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-secret
      key: DB_PASSWORD
```

---

4. Traffic Flow -- Request to Pod

```
  User's browser
       |
       |  HTTPS request
       v
  AWS ELB (Elastic Load Balancer)
       |  Created automatically when Service type=LoadBalancer
       |  Distributes across all available Kubernetes nodes
       v
  Ingress Controller  (optional but recommended)
       |  Routes based on URL path and hostname:
       |    /api     ->  api-service
       |    /admin   ->  admin-service
       |    /static  ->  static-service
       v
  Kubernetes Service
       |  Stable DNS: my-app-service.production.svc.cluster.local
       |  Load-balances across all healthy pods with matching label
       |  If a pod dies -> immediately stops sending traffic to it
       v
       +--------------+--------------+
       v              v              v
    Pod 1          Pod 2          Pod 3
   (running)      (running)      (running)
   app:8080       app:8080       app:8080
```

Ingress -- URL-based routing:

ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-app-ingress
  annotations:
    kubernetes.io/ingress.class: "alb"
    alb.ingress.kubernetes.io/scheme: "internet-facing"
spec:
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
```

5. Auto Scaling -- Two Levels

Level 1 -- HPA (Horizontal Pod Autoscaler)

Adds more **pods** when CPU or memory is high. Fast -- happens in seconds.

hpa.yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70     # scale when CPU > 70%
```

```
# Quick HPA creation
kubectl autoscale deployment my-app --min=2 --max=10 --cpu-percent=70

# Watch HPA in action
kubectl get hpa -w
```

Level 2 -- Cluster Autoscaler (AWS)

Adds more **EC2 nodes** when pods cannot be scheduled. Slower -- takes 2-5 minutes.

```
Traffic spike
     |
     v  HPA adds more pods -> all nodes full
Pods stuck in "Pending" state
     |
     v  Cluster Autoscaler detects Pending pods
Calls AWS Auto Scaling Group API: "add a node"
     |
     v  New EC2 node joins cluster (2-5 min)
Pending pods scheduled to new node -> traffic handled

Traffic drops -> HPA removes pods -> nodes underutilised
     |
     v  Cluster Autoscaler removes empty nodes -> cost savings
```

	HPA	Cluster Autoscaler
What scales	Pods (containers)	EC2 nodes (machines)
Trigger	CPU/memory/custom metric	Pods stuck in Pending
Speed	Seconds	2-5 minutes

	HPA	Cluster Autoscaler
AWS integration	Metrics from CloudWatch	Auto Scaling Groups

6. Getting Started with AWS EKS

Step 1 -- Install tools

```
# macOS
brew install awscli kubectl eksctl

# Linux -- AWS CLI
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip awscliv2.zip && sudo ./aws/install

# Linux -- kubectl
curl -LO "https://dl.k8s.io/release/$(curl -sL https://dl.k8s.io/release/stable.txt)...
sudo install kubectl /usr/local/bin/kubectl

# Linux -- eksctl
curl --silent --location "https://github.com/weaveworks/eksctl/releases/latest/downl...
sudo mv /tmp/eksctl /usr/local/bin
```

Step 2 -- Configure AWS credentials

```
aws configure
# AWS Access Key ID:     [your key]
# AWS Secret Access Key: [your secret]
# Default region:        us-east-1
# Output format:         json
```

Step 3 -- Create EKS cluster (15-20 minutes)

```
eksctl create cluster \
  --name my-cluster \
  --region us-east-1 \
  --nodegroup-name my-nodes \
  --node-type t3.small \
  --nodes 2 \
  --nodes-min 1 \
  --nodes-max 4 \
  --managed

# eksctl automatically configures kubectl -- verify:
kubectl get nodes
# ip-192-168-1-1.ec2.internal    Ready    <none>   2m
# ip-192-168-1-2.ec2.internal    Ready    <none>   2m
```

Step 4 -- Push Docker image to ECR

```
# Create ECR repository
aws ecr create-repository --repository-name my-springboot-app

export AWS_ACCOUNT=$(aws sts get-caller-identity --query Account --output text)
export ECR=$AWS_ACCOUNT.dkr.ecr.us-east-1.amazonaws.com

# Login Docker to ECR
aws ecr get-login-password --region us-east-1 | docker login --username AWS --passwo...

# Build, tag, push
mvn clean package -DskipTests
docker build -t my-springboot-app:1.0.0 .
docker tag my-springboot-app:1.0.0 $ECR/my-springboot-app:1.0.0
docker push $ECR/my-springboot-app:1.0.0
```

Step 5 -- Deploy to Kubernetes

```
# Apply all YAML files in the k8s/ folder at once
kubectl apply -f k8s/

# Watch pods start up
kubectl get pods -w

# Watch rollout complete
kubectl rollout status deployment/my-springboot-app

# Get your public URL (1-2 min to provision ELB)
kubectl get service my-springboot-app-service
# EXTERNAL-IP: abc123.us-east-1.elb.amazonaws.com

# Test it!
curl http://abc123.us-east-1.elb.amazonaws.com/actuator/health
# {"status":"UP"}
```

Update to new version -- zero downtime

```
# Build and push new image
docker build -t my-springboot-app:2.0.0 .
docker push $ECR/my-springboot-app:2.0.0

# K8s rolls pods one by one -- users never see downtime
kubectl set image deployment/my-springboot-app \
  my-springboot-app=$ECR/my-springboot-app:2.0.0

# If something breaks -- instant rollback
kubectl rollout undo deployment/my-springboot-app
```

Clean up -- avoid charges

```
# Delete all K8s resources
kubectl delete -f k8s/

# Delete the entire cluster (removes EC2 nodes, VPC, everything)
eksctl delete cluster --name my-cluster --region us-east-1
```

---

7. Essential kubectl Commands

Get information

```
kubectl get all                          # everything
kubectl get pods -o wide                 # with IPs and node
kubectl get pods -w                      # watch real time

kubectl describe pod <pod-name>          # full detail + events
kubectl describe deployment my-app

kubectl logs -f <pod-name>               # live logs
kubectl logs <pod-name> --previous       # previous crashed pod
```

Deploy and manage

```
kubectl apply -f deployment.yaml         # create or update
kubectl apply -f k8s/                    # apply whole folder
kubectl delete -f deployment.yaml        # delete resources

kubectl scale deployment my-app --replicas=5
kubectl rollout restart deployment/my-app   # force restart all pods
```

Debugging

```
# Shell into a running pod
kubectl exec -it <pod-name> -- /bin/bash

# Copy files to/from a pod
kubectl cp ./local.txt my-pod:/app/config/

# Port-forward -- access pod directly for testing
kubectl port-forward pod/my-pod 8080:8080
# Now: curl http://localhost:8080/api/health

# Resource usage (requires metrics-server)
kubectl top pods
kubectl top nodes
```

---

8. Kubernetes vs Traditional Deployment

Concern	Traditional (manual)	Kubernetes
App crash	Manual restart or alarm	Automatic restart in seconds
Scale up	Manually add EC2, configure ELB	kubectl scale --replicas=10
Deploy new version	SSH into servers, downtime	Rolling update, zero downtime
Rollback	SSH back in, restart old version	kubectl rollout undo
Config management	SSH, update files, restart	ConfigMap update + rolling restart
Service discovery	Hard-coded IPs or Route 53	Built-in DNS (service.namespace)
Multi-region	Complex manual setup	Cluster per region, same YAML

---

9. Quick Reference

Kubernetes objects

Object	Purpose
Pod	One or more containers running together
Deployment	Manages desired number of pod replicas + rolling updates
Service	Stable network endpoint -- load-balances across pods
Ingress	URL-based routing rules to services
ConfigMap	Non-sensitive configuration (ports, feature flags)
Secret	Sensitive values (passwords, API keys)
Namespace	Virtual cluster isolation (dev/staging/prod)
HPA	Auto-scale pods based on CPU/memory/custom metrics
PersistentVolumeClaim	Request EBS storage for stateful apps

kubectl cheat sheet

```
# Resource shortcuts
kubectl get po          # pods
kubectl get deploy      # deployments
kubectl get svc         # services
kubectl get ing         # ingresses
kubectl get cm          # configmaps
kubectl get secret      # secrets
kubectl get ns          # namespaces
kubectl get hpa         # horizontal pod autoscalers
kubectl get nodes       # cluster nodes

# -n flag sets namespace
kubectl get pods -n production

# -o yaml shows the full YAML of any resource
kubectl get deployment my-app -o yaml

# -o jsonpath extracts a specific field
kubectl get pod my-pod -o jsonpath='{.status.podIP}'
```

AWS EKS + Kubernetes relationship

```
AWS Account
  +-- EKS Cluster (my-cluster)
        +-- Control Plane  (managed by AWS -- $0.10/hr)
        |     +-- API Server
        |     +-- etcd
        |     +-- Scheduler
        |     +-- Controller Manager
        +-- Node Group  (my-nodes)
              +-- EC2 t3.small  Node 1   <- you pay for these
              +-- EC2 t3.small  Node 2
              +-- Auto Scaling Group (min:1, max:4)
```
